

CPI

Processador anterior: Multi-cycle (pois o LOAD demorava dois ciclos)

Considerações

Each instruction type contributes differently:

- ALU instructions (add, sub, etc.) → **1 cycle**
- Store instructions → typically **1 cycle** (assuming no extra delay)
- Branch instructions → usually **1 cycle**
- Load instructions → **2 cycles**

Cálculo do CPI:

$$\text{CPI} = \sum (\text{fraction of each instruction} \times \text{cycles for that instruction})$$

eg:

Suppose your program has:

- 70% ALU instructions → 1 cycle
- 20% load instructions → 2 cycles
- 10% store/branch → 1 cycle

Then:

$$\text{CPI} = (0.7 \times 1) + (0.2 \times 2) + (0.1 \times 1)$$

$$\text{CPI} = (0.7 \times 1) + (0.2 \times 2) + (0.1 \times 1)$$

$$\text{CPI} = 0.7 + 0.4 + 0.1 = 1.2$$

tempo de um ciclo depende do tempo que os elementos de HW demoram a atuar

Processador atual: Pipelined

Objetivo: aumentar a frequência do processador

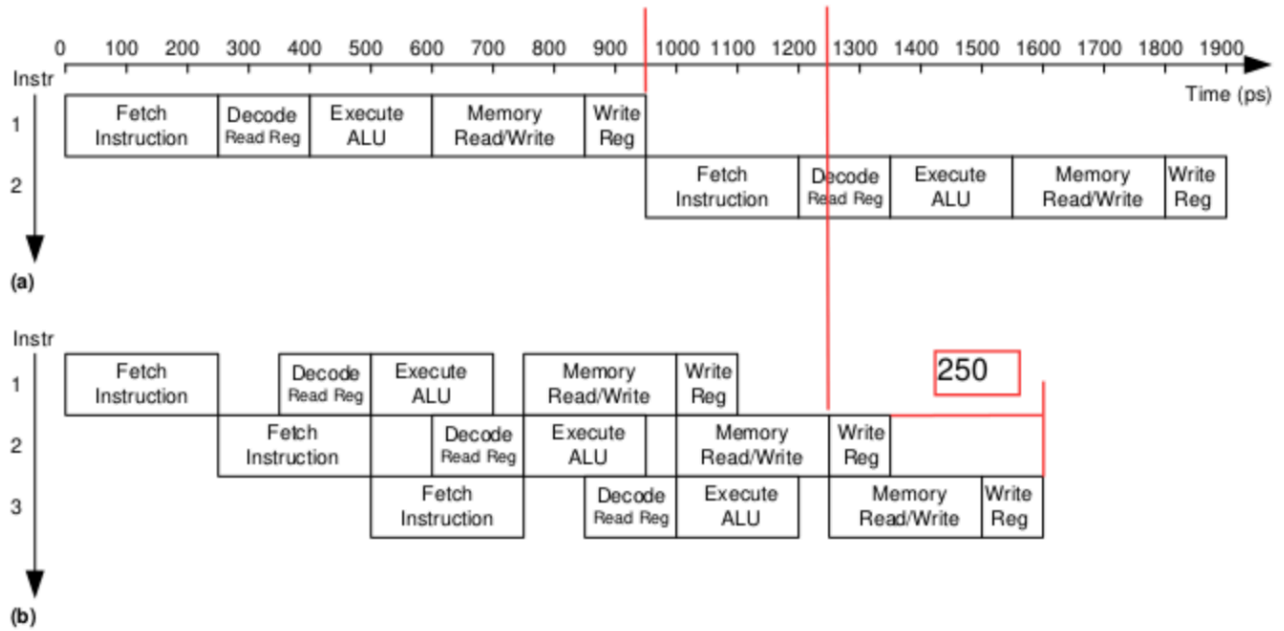


Figure 7.43 Timing diagrams: (a) single-cycle processor; (b) pipelined processor

Considerações

CPI average is ideally 1 (sai uma instrução com clk cycle)

However, a stall or a flush wastes a cycle, so the CPI is slightly higher and depends on the

specific program being executed

7.5.5 Performance Analysis

The pipelined processor ideally would have a CPI of 1, because a new instruction is issued every cycle. However, a stall or a flush wastes a cycle, so the CPI is slightly higher and depends on the specific program being executed.

Example 7.9 PIPELINED PROCESSOR CPI

The SPECINT2000 benchmark considered in Example 7.7 consists of approximately 25% loads, 10% stores, 11% branches, 2% jumps, and 52% R-type instructions. Assume that 40% of the loads are immediately followed by an instruction that uses the result, requiring a stall, and that one quarter of the branches are mispredicted, requiring a flush. Assume that jumps always flush the subsequent instruction. Ignore other hazards. Compute the average CPI of the pipelined processor.

Solution: The average CPI is the sum over each instruction of the CPI for that instruction multiplied by the fraction of time that instruction is used. Loads take one clock cycle when there is no dependency and two cycles when the processor must stall for a dependency, so they have a CPI of $(0.6)(1) + (0.4)(2) = 1.4$. Branches take one clock cycle when they are predicted properly and two when they are not, so they have a CPI of $(0.75)(1) + (0.25)(2) = 1.25$. Jumps always have a CPI of 2. All other instructions have a CPI of 1. Hence, for this benchmark, Average CPI = $(0.25)(1.4) + (0.1)(1) + (0.11)(1.25) + (0.02)(2) + (0.52)(1) = 1.15$.
